PolEasyBazaar — Full Product Requirements Document & Codex Implementation Plan

---

OVERVIEW & CONTEXT

Project Name: PolEasyBazaar **Type:** Hackathon MVP (OpenAI Codex Hackathon) **Built By:** Product Manager @ PolicyBazaar **Tagline:** "Protecting India with Insurance" **Core Idea:** A redesigned, Gen-Z-friendly wrapper over PolicyBazaar that removes friction, eliminates reliance on sales callers, and creates a self-serve insurance discovery and education experience for digitally native, first-time insurance buyers — specifically recent graduates entering the workforce.

Why This Matters: Insurance penetration in India is critically low. The primary blocker for the new generation of buyers is not awareness — it is friction. The existing PolicyBazaar experience is cluttered, sales-call-dependent, and not designed for someone who wants to self-educate and purchase independently. PolEasyBazaar is a consumer-facing wrapper that solves exactly this: a clean, guided, trust-building funnel that ends in a redirect to PolicyBazaar for actual purchase. This is not a competing product — it is a discovery and education layer on top of PolicyBazaar's existing catalog.

Constraints for Hackathon:

- No real OTP verification (phone number capture only, no actual OTP flow)

- No real checkout — all purchase CTAs redirect to the corresponding PolicyBazaar product page
- Data layer is a CSV file operated via Python/Pandas (no full database)
- Deployment via Vercel or GitHub Pages
- Scraper collects data from PolicyBazaar's public pages for: Health, Motor, Car, Term Life insurance

---

TECH STACK DECISIONS

- **Frontend:** React (with Vite), TailwindCSS for styling, Framer Motion for animations
- **Backend/Scraper:** Python with `requests`, `BeautifulSoup` / `Playwright` for scraping; `pandas` for data management; saved as CSV files
- **Search:** Client-side fuzzy search over the CSV data (use Fuse.js in React)
- **Auth (Mock):** Phone number entry stored in localStorage; no OTP for hackathon
- **Deployment:** Vercel (easiest for React/Vite projects; connect GitHub repo and deploy automatically)
- **Data Storage:** Two CSV files — `insurance_products.csv` and `insurance_faqs.csv`

Goal

Collect structured data from PolicyBazaar's public-facing pages for the following insurance categories: Health Insurance, Term Life Insurance, Car Insurance, Motor/Bike Insurance. This data will power the search bar, the FAQ education section, and the product cards.

Step 0.1 — Identify Target URLs

Before writing the scraper, manually identify and hardcode the target URLs for each insurance category from PolicyBazaar. Use the following starting points:

- `https://www.policybazaar.com/health-insurance/`
- `https://www.policybazaar.com/term-insurance/`
- `https://www.policybazaar.com/car-insurance/`
- `https://www.policybazaar.com/two-wheeler-insurance/`

Also check if PolicyBazaar exposes a sitemap at `https://www.policybazaar.com/sitemap.xml` — if yes, parse this XML to discover article/guide/product URLs automatically.

Step 0.2 — Build the Scraper Script (`scraper/scrape_pb.py`)

```
scraper/
  scrape_pb.py          ← main scraper script
  data/
```

```
insurance_products.csv
insurance_faqs.csv
```

scrape_pb.py should do the following:

1. Use `requests` with browser-like headers to avoid basic bot detection. If the site blocks simple requests, fall back to `playwright` (headless Chromium) for JavaScript-rendered pages.
2. For each insurance category page, extract:
   - Product/plan names
   - Key benefits (bullet points on the page)
   - Starting price or price range (if shown)
   - A short description
   - The direct PolicyBazaar URL for that plan (this becomes the redirect link)
3. For each category's education/article/guide pages, extract:
   - Article headings (H2, H3)
   - Paragraph content under each heading
   - Reformat this into FAQ pairs: the heading becomes the Question, the paragraph content becomes the Answer
4. Write everything to two CSV files:
   - `insurance_products.csv` — columns: `id, category, plan_name, description, key_benefits, price_range, pb_redirect_url`

- `insurance_faqs.csv` — columns: `id, category, question, answer`

Important Notes for Codex:

- Add `time.sleep(1-3)` between requests to be polite and avoid rate limiting
- Handle HTTP errors gracefully with try/except
- Clean all text (strip HTML tags, collapse whitespace, remove special characters)
- If a field is not found, store an empty string rather than crashing
- At the end, print a summary: how many products and FAQs were scraped per category

---

PHASE 1 — PROJECT SETUP

Step 1.1 — Initialize the React App

Run: `npm create vite@latest poleasybazaar -- --template react`

Then install dependencies:

```
npm install
npm install react-router-dom framer-motion fuse.js
papaparse tailwindcss autoprefixer postcss
npx tailwindcss init -p
```

Configure `tailwind.config.js` to scan `./src/**/*.{js,jsx}`.

Step 1.2 — Folder Structure

Set up the following folder structure inside `src/`:

```
src/
  components/
    Navbar.jsx
    SearchBar.jsx
    InsuranceGrid.jsx
    JourneyTimeline.jsx
    PhoneLoginModal.jsx
    FAQAccordion.jsx
    ProductCard.jsx
  pages/
    HomePage.jsx
    LearnPage.jsx
    CalculatorPage.jsx
    SearchResultsPage.jsx
  data/
    insurance_products.csv   ← copy from scraper output
    insurance_faqs.csv       ← copy from scraper output
  hooks/
    useAuth.js
    useInsuranceData.js
  App.jsx
  main.jsx
  index.css
```

Step 1.3 — Load CSV Data into React

In `useInsuranceData.js`, use `papaparse` to parse both CSV files at

app load time and store them in React state/context. Export two arrays: `products` and `faqs`. All components will consume data from here — no API calls needed.

Step 1.4 — Set Up React Router

In `App.jsx`, configure the following routes:

- `/` → `HomePage`
- `/learn/:category` → `LearnPage`
- `/calculate` → `CalculatorPage`
- `/search` → `SearchResultsPage`

All routes except `/` should be protected by the auth gate (see Phase 3).

PHASE 2 — HOMEPAGE (The Hero Experience)

This is the most important page. It must be visually striking, clean, and immediately communicate trust and simplicity.

Step 2.1 — Navbar (`Navbar.jsx`)

- Left: Logo text "PolEasyBazaar" in a bold, modern font
- Center: Search bar (described in Step 2.2)
- Right: "Login" button (triggers PhoneLoginModal if not logged in; shows phone number if logged in)
- Background: Semi-transparent with a blur/frosted glass effect, sticky on scroll

This is the "Google for Insurance" feature. Design it as a prominent, centered search input (on the navbar and also a large hero version on the homepage).

Behavior:

- On keystroke, use `Fuse.js` to fuzzy-search across `plan_name`, `description`, and `key_benefits` in the products CSV
- Debounce input by 300ms
- Show a dropdown of up to 5 results as the user types, each showing: plan name, category badge, short description
- On selecting a result, if the user is not logged in → trigger `PhoneLoginModal`; if logged in → navigate to `/search?q=<query>` and show full results
- On pressing Enter, navigate to `/search?q=<query>`

Step 2.3 — Hero Section (Top of Homepage)

Full-width hero area with:

- Centered headline: "PolEasyBazaar" in large display type
- Subheadline: "Protecting India with Insurance" in a softer weight
- Below the text: An animated graphic row showing 4 insurance type cards side by side — Health, Life, Car, Motor — each with:
    - A distinct color/glow (e.g., Health = teal glow, Life = amber glow, Car = blue glow, Motor = purple glow)

    - An icon (heart, shield, car, bike — use lucide-react or SVG)
    - Label text
    - A subtle hover animation (scale up slightly, glow intensifies)
    - On click: if not logged in → PhoneLoginModal; if logged in → navigate to `/learn/<category>`
- Background: Dark gradient with subtle animated particle/noise texture to give depth

Step 2.4 — Journey Timeline (`JourneyTimeline.jsx`)

Below the hero, a full-width section with a horizontal 3-step flow:

```
[ Learn About Insurance ] ──→ [ Calculate What You Need ]
──→ [ Buy Insurance ]
```

Each step is a styled rectangle/card with:

- Step number (01, 02, 03)
- Title
- Short 1-line description ("Understand what each plan covers", "Find the right coverage for your life", "Get redirected to purchase safely")
- A CTA button inside each card
- Connecting arrows between cards (CSS or SVG)
- On clicking any card CTA: if not logged in → PhoneLoginModal; if logged in → navigate to the appropriate page

This section communicates the entire user journey at a glance — this is the core UX innovation of the homepage.

Step 2.5 — Footer

Simple footer with: "PolEasyBazaar is a discovery tool. All purchases are completed on PolicyBazaar.com." and links to PolicyBazaar.

PHASE 3 — PHONE LOGIN MODAL (The Auth Gate)

Step 3.1 — PhoneLoginModal Component (`PhoneLoginModal.jsx`)

This modal appears whenever a non-logged-in user tries to access any protected content. It is a gate — not a registration wall. Keep it minimal and fast.

Design:

- Centered modal overlay with frosted glass background
- Headline: "One step away"
- Sub-text: "Enter your phone number to continue. No spam, no sales calls."
- Input: Phone number field (10-digit, numeric keyboard on mobile, with +91 prefix shown)
- CTA Button: "Continue"
- On submit: validate that it is a 10-digit number; if valid, store `{ phone: '<number>', loggedIn: true }` in `localStorage`;

close modal; proceed with the original action the user attempted

- No OTP step for hackathon — the phone number IS the account
- If user closes modal without submitting, they stay on the homepage

Step 3.2 — `useAuth.js` Hook

Create a custom hook that:

- Reads auth state from `localStorage` on init
- Exposes: `isLoggedIn` (boolean), `phone` (string), `login(phone)` function, `logout()` function
- `login(phone)` writes to localStorage and updates state
- All protected routes/components use this hook to check auth

---

PHASE 4 — LEARN PAGE (Education, Redesigned)

Step 4.1 — LearnPage (`LearnPage.jsx`)

This page is accessed at `/learn/:category` where category is one of: `health`, `life`, `car`, `motor`.

Layout:

- Page header: Category name + icon + a short intro sentence
- Main content: FAQ Accordion
- Sidebar or bottom section: Product cards for this category with a "View on PolicyBazaar" CTA

Step 4.2 — FAQ Accordion (`FAQAccordion.jsx`)

This replaces PolicyBazaar's long-form content pages entirely.

Behavior:

- Load all FAQs from `insurance_faqs.csv` filtered by the current category
- Render them as an accordion: each question is a clickable row; clicking it expands the answer below; only one answer open at a time (or allow multiple — designer's choice)
- Add a small search/filter bar above the accordion so users can filter FAQs by keyword within the page
- Animate open/close with smooth height transition (use Framer Motion's `AnimatePresence`)
- Each FAQ item should have a subtle left border in the category's accent color

This component is the core content innovation — all of PolicyBazaar's messy long-form educational content compressed into clean, scannable, self-serve FAQ format.

Step 4.3 — Product Cards (`ProductCard.jsx`)

For each insurance product in `insurance_products.csv` matching the current category, show a card with:

- Plan name
- Short description

- Key benefits (show up to 3 bullet points, with a "See more" toggle)
- Price range (if available)
- CTA Button: "View on PolicyBazaar" → opens the `pb_redirect_url` in a new tab

Cards should be in a 2 or 3-column grid, responsive.

PHASE 5 — CALCULATOR PAGE

Step 5.1 — CalculatorPage (`CalculatorPage.jsx`)

A simple, interactive tool to help users understand what level of coverage they might need. This is not a real underwriting calculator — it is a guidance tool.

Form inputs:

- Age (number input or slider, 18–65)
- Monthly income (dropdown ranges: <25k, 25k–50k, 50k–1L, 1L+)
- Number of dependents (0, 1, 2, 3+)
- Insurance type of interest (dropdown: Health, Life, Car, Motor)
- Do you have any existing insurance? (Yes/No toggle)

On clicking "Calculate":

- Apply simple rule-based logic (not ML) to recommend:

    - A coverage amount range (e.g., "You likely need ₹10–15L health cover")
    - A budget estimate (e.g., "Expected premium: ₹6,000–9,000/year")
    - 1–2 recommended product cards from the CSV data (filtered by category, shown as ProductCard components)
- Show results in an animated reveal section below the form
- Each recommended product has a "View on PolicyBazaar" redirect button

The logic for recommendations can be a simple if/else decision tree hardcoded in a utility function `src/utils/calculator.js`.

PHASE 6 — SEARCH RESULTS PAGE

Step 6.1 — SearchResultsPage (`SearchResultsPage.jsx`)

Accessed at `/search?q=<query>`.

Behavior:

- On mount, read the `q` param from the URL
- Run `Fuse.js` search across all products (all categories) using the query
- Display results as a vertical list of `ProductCard` components
- Show the query at the top: "Showing results for: <query>"
- If no results: show a friendly empty state with suggestions to browse by category

- Add category filter chips at the top (All, Health, Life, Car, Motor) to filter results client-side

---

PHASE 7 — STYLING & VISUAL DESIGN SYSTEM

Step 7.1 — Global Design Tokens (in `index.css` or a `theme.js`)

Define and use throughout:

```
Primary background: #0A0A0F (near black)
Card background: #13131A
Accent Health: #00E5C0 (teal)
Accent Life: #FFB347 (amber)
Accent Car: #4A9EFF (blue)
Accent Motor: #B56AFF (purple)
Text primary: #F0F0F5
Text secondary: #8888A0
Border: rgba(255,255,255,0.08)
```

Font pairing: Use Google Fonts. Display/heading: "Clash Display" or "Space Grotesk" variant that feels premium. Body: "DM Sans" or "Instrument Sans". Import via `<link>` in `index.html`.

Step 7.2 — Animation Guidelines (Framer Motion)

- Homepage hero: Staggered fade-up on load for headline, subheadline, insurance category cards (100ms delay between each)
- Insurance category cards: Hover = scale(1.04) + intensify glow

- Journey timeline cards: Fade-in on scroll (use Framer Motion's `whileInView`)
- Modal: Scale in from 0.9 + fade on mount; reverse on close
- FAQ accordion: Smooth height animation via `AnimatePresence`
- Page transitions: Fade between routes (wrap `<Routes>` in a motion component)

PHASE 8 — DEPLOYMENT

Step 8.1 — Prepare for Vercel Deployment

1. Push the entire project to a GitHub repository
2. Go to vercel.com, sign in with GitHub, click "New Project", select the repo
3. Vercel auto-detects Vite; set build command to `npm run build` and output directory to `dist`
4. Click Deploy — Vercel gives you a live URL in ~60 seconds
5. For the CSV data files: they live inside `src/data/` and are bundled at build time by Vite — no separate hosting needed

Step 8.2 — Environment Notes

- No environment variables needed for this hackathon build (no real API keys)
- The scraper is a standalone Python script run locally before deployment; its output CSVs are committed to the repo inside

`src/data/`

EXECUTION ORDER FOR CODEX

Codex should implement the above in this exact sequence, one phase at a time, confirming output before moving to the next:

```
Phase 0 → Scraper (Python) — run this first, generate CSVs
Phase 1 → Project setup, folder structure, routing, CSV loading
Phase 2 → Homepage: Navbar, Hero, InsuranceGrid, JourneyTimeline, Footer
Phase 3 → PhoneLoginModal + useAuth hook + auth gate on all routes except /
Phase 4 → LearnPage: FAQAccordion + ProductCard grid
Phase 5 → CalculatorPage: form + rule-based logic + results display
Phase 6 → SearchResultsPage: Fuse.js search + results + filter chips
Phase 7 → Global styling pass: design tokens, fonts, animations, polish
Phase 8 → Vercel deployment prep + README
```

WHAT POLEASYBAZAAR IS NOT

To keep Codex focused, explicitly call out what is out of scope:

- No real OTP or SMS verification

- No payment processing or real checkout
- No backend server or database (CSV + localStorage only)
- No admin dashboard
- No real-time pricing API
- No account management beyond phone number storage
- No comparison feature between plans (future scope)

SUCCESS CRITERIA FOR THE HACKATHON

1. A user can land on the homepage and immediately understand what PolEasyBazaar does
2. A user can search for "health insurance" and see relevant results
3. A user who clicks anything is prompted to enter their phone number (one-time)
4. A user can read FAQs about any of the 4 insurance categories without scrolling through walls of text
5. A user can use the calculator to get a coverage recommendation
6. Every product card has a working "View on PolicyBazaar" link that redirects correctly
7. The site is live on a Vercel URL and works on mobile

Document prepared for OpenAI Codex Hackathon. PolEasyBazaar — making insurance simple for the next generation of India.